Temperature Forecasting Using Time Series Analysis & Diabetic Analysis Using Logistic Regression

1st Mohammad Saad Shaikh
School of Computing, National College of Ireland
2163573 & x22163573@student.ncirl.ie

***Abstract*—The context of temperature forecasting to model past temperature data and estimate future temperature trends. The work of the Climate is a condensed time series of annual average temperatures in Armagh from 1844 to 2004 and a monthly time series of average temperatures in Armagh from January 1844 to December 2004. Forecasts for 2004 compared to the actual data. A set of risk factors (such as age, gender, and diabetes history) and the likelihood of acquiring diabetes can be modeled using logistic regression in the context of diabetes. Using a logistic regression model, diabetes can be diagnosed using blood test results. To decide how to alter the variables, use the exploratory data analysis. executing this study using Python.**

***Index Terms*—Logistic regression model, Python.**

I. Introduction

The method of projecting future temperatures using previous temperature data is known as temperature forecasting. Time series analysis and regression modeling are two examples of statistical and machine learning techniques that can be used for this. In this work, a model that can predict the average temperatures for 2004 will be developed using the data up to and including 2003 as a training set. The model's predictions will then be compared to the actual temperature data from 2004 as a test set. The next year's temperatures will be predicted using the yearly temperature data, while the following 12 months' temperatures will be predicted using the monthly temperature data. We can predict future temperature trends by performing this task of temperature forecasting in order to obtain insights into the patterns and trends in previous temperature data. This can be useful for a number of areas, including climate science, production, and agriculture.

Monthly Temperature in Armagh

Get a better understanding of the patterns and historical trends in Armagh's monthly temperature data by visualizing it.

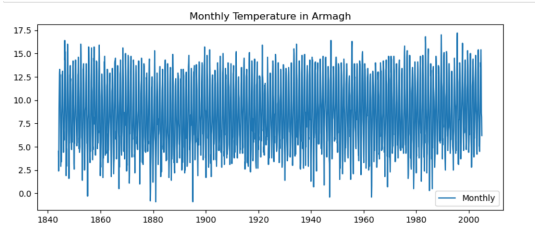


Fig. 1. Monthly Temperature in Armagh

Diabetes is a chronic disease that causes elevated blood sugar levels that, if unchecked, may trigger a number of medical conditions. Blood tests are frequently used to diagnose diabetes and to check blood sugar levels in people who already have the disease.It may be possible to find patterns or connections between specific blood test results and the chance of having diabetes by using a logistic regression model based on a large dataset of blood test results and diabetes diagnoses.A model like this would make it easier for medical professionals to identify diabetes in patients and to start preventing complications early. A model like this could help make it simpler on medical professionals to identify diabetes in patients and to commence preventing complications early. Additionally, based on the outcomes of their blood tests, it could potentially be used as a tool to help individuals determine their risk for getting diabetes.

Distribution plots for continuous variables in the diabetes data set such as AGE,Urea,Cr, HbA1, Chol,TG, HDL, LDL, VLDL, BMI.

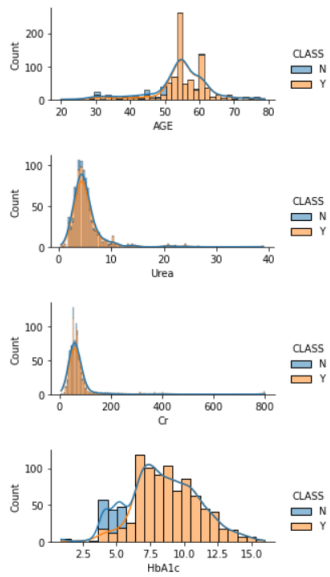


Fig. 2. Distribution plots for continuous variables

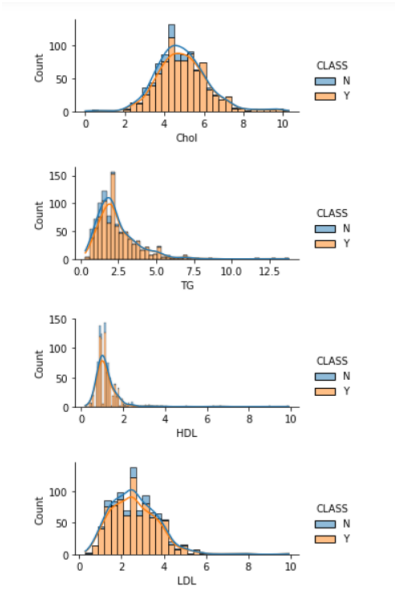


Fig. 3. Distribution plots for continuous variables

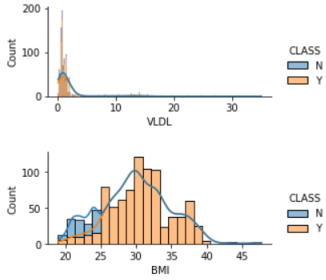
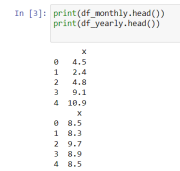


Fig. 4. Distribution plots for continuous variables

II. METHODOLOGY

A.Time Series Analysis



```
In [3]: print(df_monthly.head())
        print(df_yearly.head())

      x
0   4.5
1   2.4
2   4.8
3   9.1
4  10.9
     x
0  8.5
1  8.3
2  9.7
3  8.9
4  8.5
```

Fig. 5. Monthly & Yearly

a) Using the Python pandas package, I imported yearly and monthly temperature data from two separate CSV files. The "nitm18442004.csv" file contains monthly data, and the "nity18442004.csv" file contains year data.:

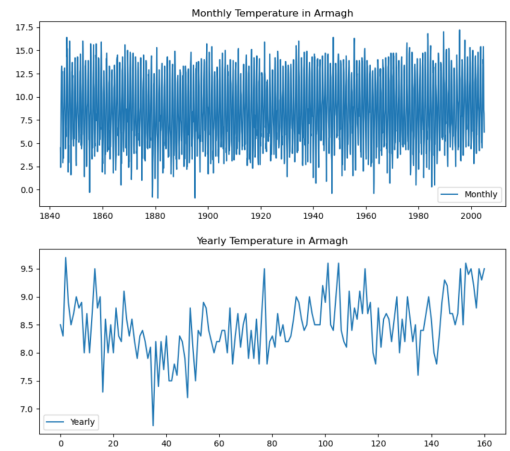


Fig. 6. Monthly & Yearly temperature in Armagh

b) Leveraging the date-range function of pandas, we are defining a date range for the monthly data. The start date is set to Jan 1st, 1844, and the end date is set to Dec 1st, 2004. The "month start" symbol (MS) that we are using to set the frequency means that the date range should include the first day of the month.The date-range function returns a pandas Datetime-Index object with a frequency of one month that contains all the dates that fall inside the specified range. The index for the monthly temperature data will be this Datetime-Index.:

c) This code analyzes down the monthly data into its trend, seasonality, and residuals components using the seasonal-decompose function from the module. The period parameter is set to 12 to denote monthly seasonality, and the parameter that describes the model is set to 'additive' to indicate that its components are cumulative rather than multiplicative.Combining subplots and plot functions from the matplotlib library, the resulting decomposition is plotted in a 4-panel figure. The observed data appears in the top panel, followed by the pattern factor in the second panel, the seasonal component in the third panel, and the residuals in the bottom panel. Each panel's y-axis is marked with a label using the

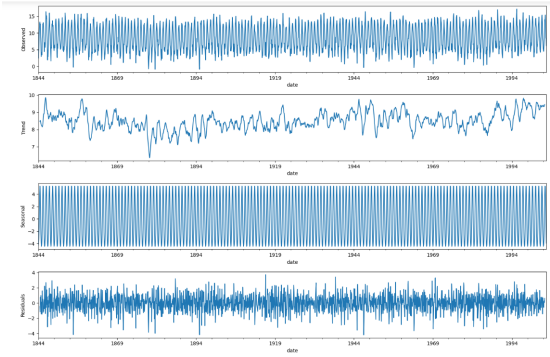


Fig. 7. Monthly & Yearly temperature in Armagh

set-ylabel method. Last but not least, tight-layout is invoked to modify the spacing. :

```
In [13]:
print('RMSE for ARIMA on yearly data: {:.2f}'.format(rmse_arima))
print('RMSE for SARIMA on yearly data: {:.2f}'.format(rmse_sarima))
print('RMSE for Exponential Smoothing on yearly data: {:.2f}'.format(rmse_es))

RMSE for ARIMA on yearly data: 0.33
RMSE for SARIMA on yearly data: 0.31
RMSE for Exponential Smoothing on yearly data: 0.31
```

Fig. 8. ARIMA/SARIMA & Exponential Smoothing

```
Forecast for 12 months using SARIMA model for monthly data: 2005-01-01     4.915573
2005-02-01     4.427701
2005-03-01     6.331934
2005-04-01     8.570938
2005-05-01    10.354430
2005-06-01    13.602182
2005-07-01    14.426449
2005-08-01    15.409927
2005-09-01    13.056819
2005-10-01     8.558862
2005-11-01     7.903729
2005-12-01     5.648598
Freq: MS, Name: predicted_mean, dtype: float64
```

Fig. 9. Forecast for 12 months using SARIMA model for monthl

```
#Forecasting for 1 year using the SARIMA model for yearly data
model_sarima_yearly = SARIMAX(df_yearly, order=(2, 1, 1), seasonal_order=(1, 1, 1, 12))
fit_sarima_yearly = model_sarima_yearly.fit()
forecast_sarima_yearly = fit_sarima_yearly.forecast(steps=1)
print('Forecast for 1 year using SARIMA model for yearly data:', forecast_sarima_yearly)

Forecast for 1 year using SARIMA model for yearly data: 161    9.317514
dtype: float64
```

Fig. 10. Forecasting for 1 year using the SARIMA model for yearly datal

d) A time series forecasting technique known as exponential smoothing gives each observation in the series a weight that decreases exponentially with time. The weights given to each observation reflect the underlying tenet that more current observations are relevant than older ones. Both trended and seasonal time series can be analyzed using this method. :

```
[34]: #Evaluating the forecasts against the actual data for 2004
actual_yearly = df_yearly[-1:]
print('Actual yearly temperature data for 2004:', actual_yearly)

Actual yearly temperature data for 2004:        x
160  9.5
```

```
[35]: #Evaluating the forecasts against the actual data for 2004
actual_monthly = df_monthly[-12:]
print('Actual monthly temperature data for 2004:', actual_monthly)

Actual monthly temperature data for 2004:                x
date
2004-01-01   5.1
2004-02-01   4.5
2004-03-01   6.2
2004-04-01   8.3
2004-05-01  10.7
2004-06-01  14.0
2004-07-01  13.6
2004-08-01  15.4
2004-09-01  13.2
2004-10-01   8.6
2004-11-01   8.2
2004-12-01   6.2
```

Fig. 11. Evaluating the forecasts yearly & monthly

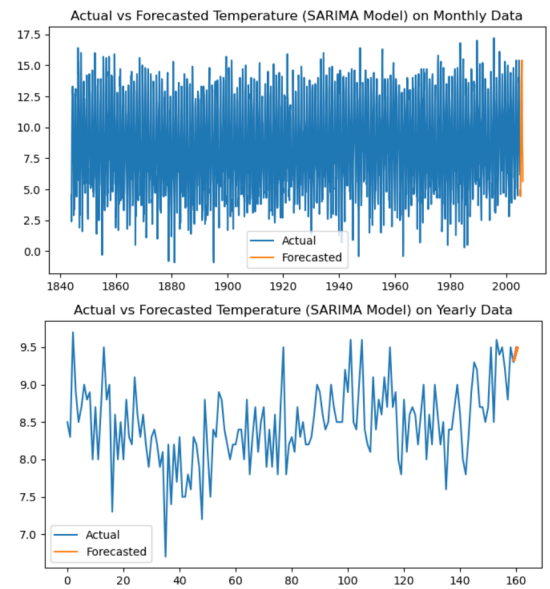


Fig. 12. Forecasts yearly & monthly

B. Logistic Regression

For binary classification issues, the common approach for machine learning is logistic regression. Logistic regression can be used in an emergency room to forecast a patient's chances of being admitted or discharged based on their medical history, vital signs, and other pertinent variables.

```
Gender    0
AGE       0
Urea      0
Cr        0
HbA1c     0
Chol      0
TG        0
HDL       0
LDL       0
VLDL      0
BMI       0
CLASS     0
dtype: int64
              AGE        Urea          Cr       HbA1c        Chol          TG  \
count  947.000000  947.000000  947.000000  947.000000  947.000000  947.000000
mean    54.101373    5.159074   69.103485    8.408617    4.878691    2.362101
std      8.499612    2.975024   60.862961    2.544040    1.313356    1.417275
min     20.000000    0.500000    6.000000    0.900000    0.000000    0.300000
25%     51.000000    3.700000   48.000000    6.800000    4.000000    1.500000
50%     55.000000    4.600000   60.000000    8.100000    4.800000    2.000000
75%     59.000000    5.700000   73.000000   10.200000    5.600000    2.900000
max     79.000000   38.900000  800.000000   16.000000   10.300000   13.800000

             HDL         LDL        VLDL         BMI
count  947.000000  947.000000  947.000000  947.000000
mean     1.209081    2.616304    1.903485   29.893897
std      0.672423    1.127316    3.757012    4.869852
min      0.200000    0.300000    0.100000   19.000000
25%      0.900000    1.800000    0.700000   27.000000
50%      1.100000    2.500000    1.000000   30.000000
75%      1.300000    3.300000    1.500000   33.000000
max      9.900000    9.900000   35.000000   47.750000
```

Fig. 13. Check for missing values

Boxplots are an effective technique to see how a variable is distributed and spot any outliers or extreme values. A multiple linear regression model's independent variables would be represented as boxplots for the data.

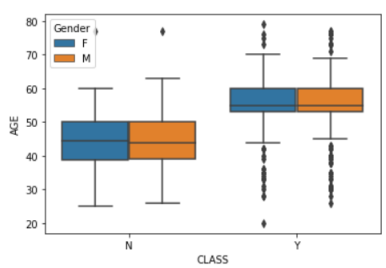


Fig. 14. Boxplots

Continuous variables with skewed distributions can be improved to make them more appropriate for analysis.

```
#Confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print(conf_matrix)

[[ 14   6]
 [  3 167]]

#Calculate the accuracy of the model
accuracy = (conf_matrix[0, 0] + conf_matrix[1, 1]) / np.sum(conf_matrix)
print("Accuracy: ", accuracy)

Accuracy:  0.9526315789473684

#Calculate the precision and recall of the model
precision = conf_matrix[1, 1] / (conf_matrix[0, 1] + conf_matrix[1, 1])
recall = conf_matrix[1, 1] / (conf_matrix[1, 0] + conf_matrix[1, 1])
print("Precision: ", precision)
print("Recall: ", recall)

Precision:  0.9653179190751445
Recall:  0.9823529411764705

#Calculate the F1 score of the model
f1_score = 2 * (precision * recall) / (precision + recall)
print("F1 score: ", f1_score)

F1 score:  0.9737609329446064

#Calculate the AUC-ROC score of the model
from sklearn.metrics import roc_auc_score
auc_roc = roc_auc_score(y_test, logreg.predict_proba(X_test)[:, 1])
print("AUC-ROC score: ", auc_roc)

AUC-ROC score:  0.9911764705882352
```

Fig. 15. Confusion matrix, accuracy, precision, recall, F1 score, AUC-ROC score for the model

```
const     179.701658
Gender      1.150187
AGE         1.317353
Urea        1.684408
Cr          1.815295
HbA1c       1.357251
Chol        1.500632
TG          1.440911
HDL         1.106333
LDL         1.347613
VLDL        1.397321
BMI         1.365036
dtype: float64
```

Fig. 16. Multicollinearity

```
Optimization terminated successfully.
         Current function value: inf
         Iterations 10
                           Logit Regression Results
==============================================================================
Dep. Variable:                  CLASS   No. Observations:                  757
Model:                          Logit   Df Residuals:                      746
Method:                           MLE   Df Model:                           10
Date:                Mon, 08 May 2023   Pseudo R-squ.:                     inf
Time:                        01:23:14   Log-Likelihood:                   -inf
converged:                       True   LL-Null:                          0.0000
Covariance Type:            nonrobust   LLR p-value:                       1.000
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Gender         1.2111      0.475      2.548      0.011       0.279       2.143
AGE            0.0176      0.024      0.740      0.459      -0.029       0.064
Urea           1.9795      0.634      3.120      0.002       0.736       3.223
Cr            -4.5088      0.647     -6.968      0.000      -5.777      -3.240
HbA1c          1.3588      0.208      6.542      0.000       0.952       1.766
Chol           0.3505      0.217      1.615      0.106      -0.075       0.776
TG             1.7466      0.506      3.450      0.001       0.754       2.739
HDL           -0.6671      0.368     -1.813      0.070      -1.388       0.054
LDL           -0.4043      0.260     -1.554      0.120      -0.914       0.106
VLDL           0.8881      0.409      2.173      0.030       0.087       1.689
BMI            0.2994      0.065      4.584      0.000       0.171       0.427
==============================================================================

Possibly complete quasi-separation: A fraction 0.41 of observations can be
perfectly predicted. This might indicate that there is complete
quasi-separation. In this case some parameters will not be identified.
```

Fig. 17. linearity

```
#Calculate the probability that the "P" class cases are diagnosed as diabetic
prob_diabetic = np.mean(prob_prediabetic)
print("Probability of diagnosing the 'P' class cases as diabetic: ", prob_diabetic)

Probability of diagnosing the 'P' class cases as diabetic:  0.9768730259460784
```

Fig. 18. Probability that the "P" class cases are diagnosed as diabetic

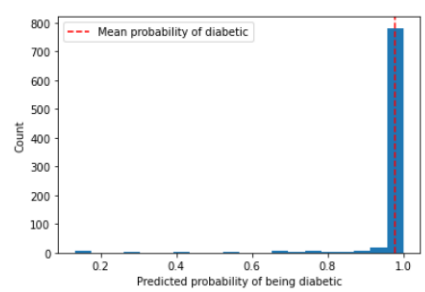


Fig. 19. Predicted probability of being diabetic

III. Conclusion

A. Time Series Analysis

The expected monthly temperature data for 2005 shows an increasing trend from January to August, with a peak in August at 15.41 degrees Celsius, before declining in the remaining months, according to projections from the SARIMA model. All of the predicted temperatures from September through December are below 14 degrees Celsius. SARIMA predicts a temperature of 9.32 degrees Celsius for 2004, which is just a little higher than the actual temperature of 9.5 degrees Celsius for the year. This can be a sign that not all the variables influencing yearly temperature swings are being adequately represented by the model.

B. Logistic Regression

It seems there is a high likelihood of diagnosing ”P” class cases as diabetes based on the probability calculation. It is crucial to remember that this likelihood was determined using the prediction model, and its precision may be dependent on the model’s quality and the data used to train it. However, if the model is well-designed and the training data are representative of the community, this probability could be a helpful predictor of the propensity for a ’P’ class case to have diabetes.

IV. References

1 https://www.who.int/news-room/fact-sheets/detail/cancer

2 https://www.investopedia.com/terms/d/descriptivestatistics.asp

3 https://www.codecademy.com/learn/learn-linear-regression-inr/modules/linear-regression-in-r/cheatsheet

4 Rashid, Ahlam (2020), “Diabetes Dataset”, Mendeley Data, V1, doi: 10.17632/wj9rwkp9c2.1